

Embedding-Guided Tool Retrieval for In-Car Voice Agents: CAR-bench IJCAI-ECAI 2026 Competition — Track 1 Technical Report

Claudia Dresselhaus
claudiadresselhaus@gmail.com

Abstract

We present our Track 1 submission to the CAR-bench Challenge at IJCAI-ECAI 2026. Our agent uses **ChatGPT 4.5 nano** [OpenAI, 2025] for multi-turn tool-calling in an automotive in-car voice assistant domain. We develop a *retrieval-guided pipeline* that pre-selects relevant tools from the 58-tool knowledge base before each task through four stages: (1) LLM-based query understanding, (2) embedding-based entity resolution, (3) LLM disambiguation of candidate tools, and (4) query-plan generation. The pipeline narrows the agent’s tool context at each task, reducing selection burden and supporting multi-step planning.

1 Introduction

CAR-bench [Kirmayr *et al.*, 2026] evaluates the epistemic reliability of multi-turn, tool-using LLM agents in an automotive in-car voice assistant domain. Three complementary task types probe different failure modes: *base* tasks require multi-step tool use and policy compliance; *disambiguation* tasks require the agent to elicit missing information before acting; *hallucination* tasks present deliberately unsatisfiable requests to test limit-awareness and refusal. Agents interact with 58 interconnected tools spanning navigation, vehicle control, charging, and productivity, governed by 19 domain-specific policies.

A central challenge is the large and heterogeneous tool space. Providing all 58 tool schemas at every turn places a heavy selection burden on the model and inflates context size. We address this with a *retrieval-guided pipeline* that narrows the candidate tool set before execution, inspired by retrieval-augmented generation [Lewis *et al.*, 2020] and structured agent planning [Yao *et al.*, 2023].

2 System Description

2.1 Baseline Tool-Calling Agent

The ToolCallingAgent drives ChatGPT 4.5 nano via LiteLLM [Berri *et al.*, 2023] with the standard OpenAI-compatible function-calling interface. All 58 tool definitions are included in the context at every turn. The agent follows a standard ReAct-style loop [Yao *et al.*, 2023]: at each turn it

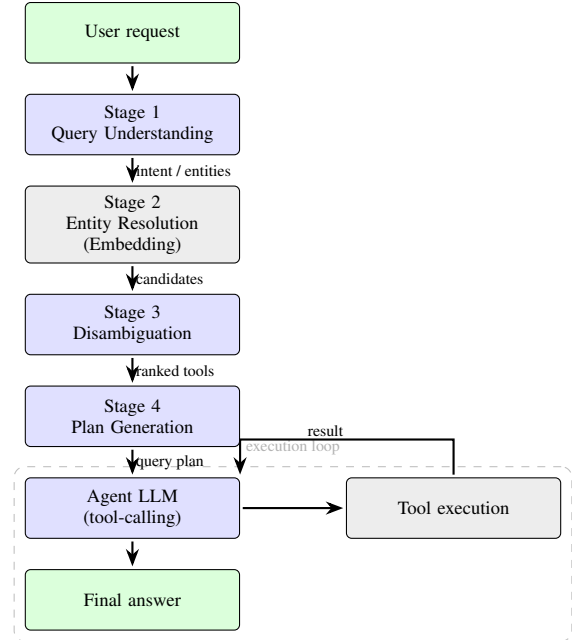


Figure 1: Architecture of the retrieval-guided pipeline. Stages 1, 3, and 4 are sequential LLM calls. Stage 2 uses embedding similarity (BGE-small) to retrieve candidate tools. Blue = LLM calls; grey = deterministic; green = I/O. The dashed box is the standard ReAct execution loop.

either emits a tool call or produces a final answer. Temperature is set to 0.0; thinking is disabled.

The system prompt encodes the 19 domain policies, tool usage guidelines, and output format constraints (metric units, 24-hour time, speech-friendly text without markdown). Three cross-domain reasoning tools (`think`, `calculate_math`, `calculate_datetime`) are always provided to support multi-step planning.

2.2 Retrieval-Guided Pipeline

The RetrievalPipelineAgent extends the baseline with four pre-execution stages that run once per task (Figure 1). If any stage raises an exception the system falls back to plain ToolCallingAgent initialisation, ensuring no hard failures.

Stage 1: Query Understanding. An LLM call parses the user request into three structured fields: *intent* (short phrase), *entities* (e.g. vehicle components, destinations, contacts), and *aspects* (qualifiers such as values, conditions, or units).

Stage 2: Entity Resolution. Embedding similarity over the full tool catalog retrieves the most relevant candidate tools for the current request (see Section 2.3).

Stage 3: Disambiguation. A second LLM call receives the candidate tool catalog and ranks the tools by relevance to the user request. The output is a JSON array of tool names validated against the candidate set.

Stage 4: Plan Generation. A third LLM call produces a concise, ordered list of tool-call steps. The plan is prepended to the agent’s initial state as a structured QUERY PLAN message that guides the execution loop.

2.3 Embedding-Based Tool Retrieval

The `DynamicGraphToolRetriever` implements embedding-based retrieval with optional graph expansion. Tool texts (name, description, and parameter names concatenated) are encoded with BAAI/bge-small-en-v1.5 [BAAI, 2024] via Sentence Transformers [Reimers and Gurevych, 2019]; embeddings are precomputed at initialisation and shared across threads. At query time, cosine similarity ranks the full catalog and the top- K candidates are selected.

In *graph* mode, BFS over the tool ontology graph adds transitive prerequisites (`semantic_req`), informing predecessors (`semantic_inf`), and routing alternatives (`routing`). The graph contains 58 nodes and 509 edges across five relation types. In *basic* mode no graph expansion is performed. An optional *labeled* mode injects policy-hint annotations into tool descriptions before embedding, improving retrieval precision for constraint-sensitive tools.

3 Experimental Setup

Models. The agent is ChatGPT 4.5 nano [OpenAI, 2025] accessed via the OpenAI API. The user simulator is also ChatGPT 4.5 nano in user-simulation mode with internal chain-of-thought.

Tasks and evaluation. We evaluate on base, disambiguation, and hallucination tasks from the CAR-bench public splits [Kirmayr *et al.*, 2026], running each task for multiple independent trials. Primary metrics are $\text{Pass}@k$ (probability that at least one of k trials succeeds) and $\hat{\text{Pass}}^k$ (all k succeed), following the unbiased estimators defined in the benchmark.

Retrieval configuration. We evaluate two retrieval modes (*basic* and *graph*) crossed with two annotation settings (*none* and *labeled*), using a retrieval depth of $K=10$ candidate tools. All inference runs on the OpenAI commercial API; the embedding model runs locally on CPU with no GPU required.

4 Conclusion

We described a Track 1 submission to CAR-bench that pairs ChatGPT 4.5 nano with a four-stage retrieval-guided pipeline. Embedding-based tool pre-selection narrows the 58-tool context before each ReAct execution loop, reducing selection burden and enabling structured multi-step planning. The pipeline’s modular design — with graceful fallback at each stage — makes it robust to edge cases and easy to extend with richer retrieval signals in future work.

References

- [BAAI, 2024] BAAI. BAAI/bge-small-en-v1.5: Embedding model. <https://huggingface.co/BAAI/bge-small-en-v1.5>, 2024.
- [Berri *et al.*, 2023] Krrish Berri, Ishaan Zhao, and Hernan Villanueva. LiteLLM: A unified interface for LLM providers. <https://github.com/BerriAI/litellm>, 2023.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.
- [Lewis *et al.*, 2020] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [OpenAI, 2025] OpenAI. ChatGPT 4.5 nano. <https://openai.com>, 2025.
- [Reimers and Gurevych, 2019] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.